

The Go Programming Language

NEDO-OFFSEC

TABLE OF CONTENT

I	Structures.....	7
II	Structures.....	9
	1. Names.....	9
	a. Ключевые слова Go.....	9
	2. Declarations.....	10
	3. Variables.....	11
	4. Assignments.....	12
	5. Type Declarations.....	13
	6. Packages and Files.....	14
	7. Scope.....	15
III	Data-Types.....	16
IV	Data-Types.....	18
	1. Integers.....	18
	a. Операции.....	18
	2. Floating-Point Numbers.....	20
	3. Complex Numbers.....	22
	4. Booleans.....	24
	5. Strings.....	25
	6. Constants.....	26
V	Interfaces.....	27
VI	Interfaces.....	29
	1. Interfaces as Contracts.....	29
	2. Interface Types.....	32
	a. Важные интерфейсы из пакета io.....	32
	b. Встраивание интерфейсов (Interface Embedding).....	32
	c. Комбинация стилей.....	32
	d. Правила встраивания.....	32
	e. Что важно запомнить.....	32
	f. Пример: fmt.Fprintf и io.Writer.....	32
	3. Interface Satisfaction.....	33
	a. Тонкий момент: Получатель (Pointer vs Value).....	33
	b. Композиция интерфейсов.....	33
	c. Пустой интерфейс interface{ }.....	33
	4. flag.Value.....	34
	a. Суть.....	34
	b. Пример: Свой флаг для температуры.....	34
	c. Как это работает.....	34
	d. Функция-конструктор.....	34
	e. Использование.....	34
	f. Запуск.....	34
	5. Ключевая мысль.....	34
	6. Interface Values.....	35
	a. Устройство интерфейса.....	35
	b. Нулевое значение интерфейса (nil).....	35

c.	Присваивание конкретного типа интерфейсу.....	35
d.	Присваивание другого типа.....	35
e.	Значение интерфейса равно nil только когда оба компонента равны nil.....	35
f.	Сравнение интерфейсов.....	35
g.	Как узнать тип внутри интерфейса.....	35
h.	Итог.....	35
7.	Sorting with sort.Interface.....	36
a.	Суть.....	36
b.	Как это работает.....	36
c.	Пример: срез строк.....	36
d.	Сортировка структуры.....	36
e.	Обратная сортировка.....	36
f.	Многоуровневая сортировка.....	36
g.	Готовые функции.....	37
h.	Ключевая мысль.....	37
8.	HTTP Handlers.....	38
a.	Суть.....	38
b.	Простейший сервер.....	38
c.	Маршрутизация по URL.....	38
d.	Использование ServeMux (мультиплексор).....	38
e.	Функция как обработчик (http.HandlerFunc).....	38
f.	Глобальный DefaultServeMux.....	39
g.	Важно!.....	39
h.	Ключевая мысль.....	39
9.	Error Interface.....	40
a.	Суть.....	40
b.	Как создать ошибку.....	40
c.	Почему там указатель, а не значение?.....	40
d.	Форматирование ошибок.....	40
e.	Пример: системная ошибка.....	40
f.	Ключевая мысль.....	40
10.	Type Assertions.....	41
a.	Суть.....	41
b.	Два варианта.....	41
i.	Вариант 1: Конкретный тип.....	41
ii.	Вариант 2: Интерфейс.....	41
c.	Безопасная форма (с проверкой).....	41
d.	Пример: распознавание ошибок.....	41
e.	Ключевая мысль.....	41
11.	Type Switches.....	42
a.	Два стиля интерфейсов.....	42
b.	Пример: sqlQuote.....	42
c.	Синтаксис.....	42
d.	Как это работает.....	42
e.	Порядок важен.....	42
f.	Где используется.....	42
g.	Ключевая мысль.....	43
VII	Goroutines and channels.....	44
VIII	Goroutines and channels.....	46

	1. Goroutines.....	46
	a. Что это.....	46
	b. Пример: анимация во время вычислений.....	46
	c. Что здесь происходит.....	46
	d. Важно.....	46
	e. Синтаксис.....	46
	f. Ключевая мысль.....	47
	2. EXAMPLE: Concurrent echo-server.....	48
	a. Простейший эхо-сервер (синхронный).....	48
	b. Эхо с задержкой (синхронный).....	48
	c. Параллельный эхо-сервер.....	48
	d. Клиент.....	48
	e. Важно!.....	48
	f. Ключевая мысль.....	48
	3. Channels.....	49
	a. Основные операции.....	49
	b. Небуферизованный канал.....	49
	c. Буферизованный канал.....	49
	d. Закрытие канала.....	49
	e. Однонаправленные каналы.....	49
	f. Конвейер (pipeline).....	49
	g. Итог.....	49
	4. Select.....	50
	a. Как работает.....	50
	b. Пример: таймер и отмена.....	50
	c. Таймеры.....	50
	d. Неблокирующий select (с default).....	50
	e. nil-каналы.....	50
	f. Ключевая мысль.....	51
	5. Cancellation.....	52
	a. Проблема.....	52
	b. Решение: закрытие канала.....	52
	c. Проверка отмены.....	52
	d. Пример: отмена по нажатию Enter.....	52
	e. Обработка отмены в цикле.....	52
	f. Как это работает.....	52
	g. Ключевая мысль.....	52
	6. EXAMPLE: Chat Server.....	53
	a. Архитектура.....	53
	b. Каналы.....	53
	c. Broadcaster.....	53
	d. Обработка клиента.....	53
	e. Как это работает.....	53
	f. Ключевая мысль.....	54
IX	Concurrency.....	55
X	Concurrency.....	57
	1. Data Races.....	57
	a. Что такое состояние гонки (race condition).....	57
	b. Гонка данных (data race).....	57
	c. Пример: банковский счёт.....	57

d. Как это выглядит.....	57
e. Три способа избежать гонки.....	57
f. Пример: ограничение через каналы.....	57
g. Ключевая мысль.....	58
2. Mutual Exclusion (<code>sync.Mutex</code>).....	59
a. Проблема.....	59
b. Решение: <code>sync.Mutex</code>	59
c. Критический раздел (critical section).....	59
d. <code>defer</code> для <code>Unlock()</code>	59
e. Мьютекс не реентерабельный.....	59
f. Паттерн: монитор.....	59
g. Ключевая мысль.....	59
3. Read/Write Mutex (<code>sync.RWMutex</code>).....	60
a. Проблема.....	60
b. Решение <code>sync.RWMutex</code>	60
c. Правила.....	60
d. Когда использовать.....	60
e. Важно.....	60
f. Ключевая мысль.....	60
4. Memory Synchronization.....	61
a. Проблема.....	61
b. Пример: недетерминированное поведение.....	61
c. Почему так происходит.....	61
d. Решение.....	61
e. Правила.....	61
f. Ключевая мысль.....	61
5. <code>sync.Once</code>	62
a. Проблема.....	62
b. Решение: <code>sync.Once</code>	62
c. Как работает <code>Do</code>	62
d. Почему лучше, чем мьютекс.....	62
e. Итог.....	62
6. Race Detector.....	63
a. Что это.....	63
b. Как использовать.....	63
c. Как работает.....	63
d. Пример отчёта.....	63
e. Ограничения.....	63
f. Ключевая мысль.....	63
7. Concurrent Cache.....	64
a. Проблема.....	64
b. Эволюция дизайна.....	64
i. Версия 1: Простой кеш (небезопасный).....	64
ii. Версия 2: С мьютексом (безопасно, но медленно).....	64
iii. Версия 3: Двойная блокировка (быстрее, но дублирует работу)... ..	64
iv. Версия 4: Подавление повторений (без дублирования).....	64
c. Монитор (управляющая горутина).....	65
d. Ключевая мысль.....	65
8. Goroutines vs Threads.....	66
a. Стеки.....	66

- b. Планирование..... 66
- c. GOMAXPROCS..... 66
- d. Идентификация горутин..... 66
- e. Ключевая мысль..... 66

The Go Programming Language

Structures

Go. Chapter II: Program Structure

NEDO-OFFSEC

Go. Chapter II: Program Structure

Structures

Names

Имена всего следуют правилу:

- 1. Имя начинается с буквы или подчеркивания
- 2. Имена чувствительны к регистру

Go имеет 25 ключевых слов:

Ключевые слова Go

Здесь перечислены основные ключевые слова языка:

break	case	chan	const	continue
default	defer	else	fallthrough	for
func	go	goto	if	import
interface	map	package	range	return
select	struct	switch	type	var

А также есть около 3 десятков предопределенных имен:

Константы:	true, false, iota, nil
Типы:	int, int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64, float32, float64, complex64, complex128, bool, byte, rune, string, error,
Функции:	make, len, cap, new, append, copy, close, delete, complex, real, imag, panic, recover

Их можно использовать в объявлениях, но будьте аккуратны, тк переобъявление может привести к путанице

Declarations

Объявление именуется программная сущность и определяет некоторые или все ее свойства

Имеется 4 основные разновидности объявлений:

- var
- const
- type
- func

Программа Go хранится в одном или нескольких файлах, имена которых заканчиваются на .go. Каждый файл начинается с объявления package, которое говорит, частью какого пакета является данный файл. За объявлением package следуют любые

объявления import, а затем — последовательность объявления типов, переменных, констант и функций уровня пакета в произвольном порядке. Например, приведенная ниже программа объявляет константу, функцию и пару переменных:

```
package main

import "fmt"

const boilingF = 212.0

func main() {
    var f = boilingF
    var c = (f - 32) * 5 / 9
    fmt.Printf("Температура кипения = %g F или %g C\n", f, c)
}
```

Variables

Assignments

Type Declarations

Packages and Files

Scope

Data-Types

Go. Chapter III: Fundamental Data Types

NEDO-OFFSEC

Go. Chapter III: Fundamental Data Types

Data-Types

Integers

Числовые типы данных Go включают целые числа нескольких размеров, числа с плавающей точкой и комплексные числа.

Go обеспечивает как знаковую, так и беззнаковую целочисленную арифметику.

Тип	Запись	Размер
signed int	int8, int16, int32, int64	8, 16, 32, 64 бита
unsigned int	uint8, uint16, uint32, uint64	8, 16, 32, 64 бита

Также есть `int` и `uint`, они имеют естественный или наиболее эффективный размер для знаковых и беззнаковых целых чисел на конкретной платформе.

Тип `rune` является синонимом для типа `int32` и указывает, что данное значение является символом Unicode. Аналогично тип `byte` является синонимом для типа `uint8` и подчеркивает, что это значение является фрагментом неформатированных данных, а не малым числом.

Наконец имеется беззнаковый целочисленный тип `uintptr`, ширина которого не указана, но достаточна для хранения всех битов значения указателя. Данный тип полезен для низкоуровневого программирования, такого, как стыковка Go с библиотекой C или с операционной системой.

Операции

Операция	Описание
<code>==</code>	Равно
<code>!=</code>	Не равно
<code><</code>	Меньше
<code><=</code>	Меньше или равно
<code>></code>	Больше
<code>>=</code>	Больше или равно

Имеются также унарные операторы `+` и `-`:

Операция	Описание
<code>+</code>	Унарный знак плюс
<code>-</code>	Унарный знак минус

Также есть побитовые операторы:

Операция	Описание
&	Побитовое И
	Побитовое ИЛИ
^	Побитовое исключающее ИЛИ (XOR)
&^	Сброс бита (И НЕ)
<<	Сдвиг влево
>>	Сдвиг вправо

Floating-Point Numbers

Go предоставляет два варианта чисел с плавающей точкой разного размера:

- float32
- float64

Предельные значения этих типов можно найти в пакете math:

- math.MaxFloat32: примерно $3.4e38$
- math.MaxFloat64: примерно $1.8e308$

А данный код создает следующую красоту:

```
package main

import (
    "fmt"
    "math"
)

const (
    width, height = 600, 320           // Размер канвы в пикселях
    cells         = 100                // Количество ячеек сетки
    xyrange       = 30.0               // Диапазон осей (-xyrange..+xyrange)
    xyscale       = width / 2 / xyrange // Пикселей в единице x или y
    zscale        = height * 0.4        // Пикселей в единице z
    angle         = math.Pi / 6         // Углы осей x, y (=30°)
)

var sin30, cos30 = math.Sin(angle), math.Cos(angle) // sin(30°), cos(30°)

func main() {
    fmt.Printf("<svg xmlns='http://www.w3.org/2000/svg' "+
        "style='stroke: grey; fill: white; stroke-width: 0.7' "+
        "width='%d' height='%d'>", width, height)

    for i := 0; i < cells; i++ {
        for j := 0; j < cells; j++ {
            ax, ay := corner(i+1, j)
            bx, by := corner(i, j)
            cx, cy := corner(i, j+1)
            dx, dy := corner(i+1, j+1)
            fmt.Printf("<polygon points='%g,%g %g,%g %g,%g %g,%g' />\n",
                ax, ay, bx, by, cx, cy, dx, dy)
        }
    }
    fmt.Println("</svg>")
}

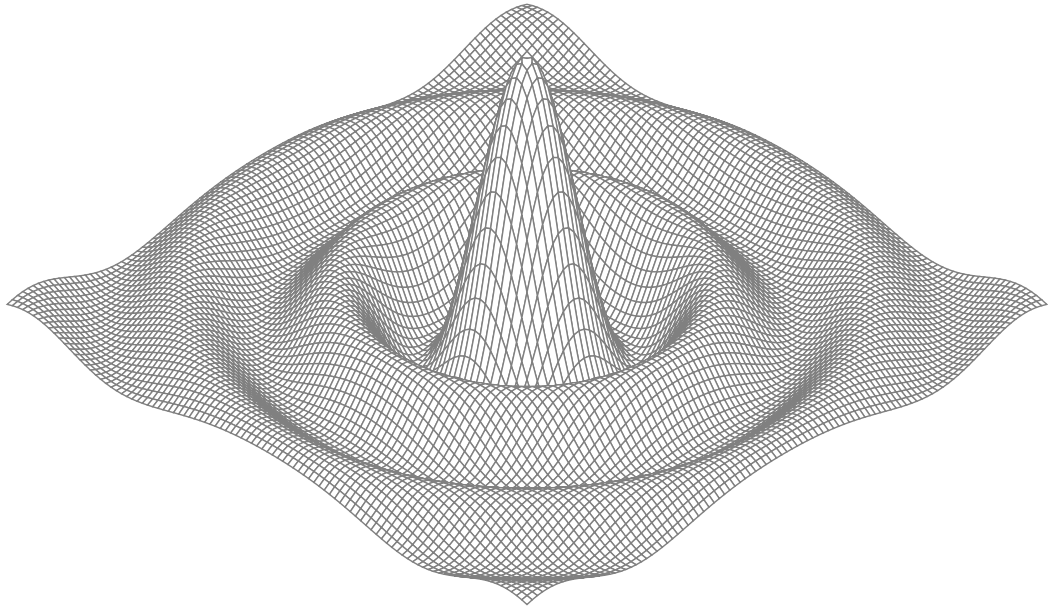
func corner(i, j int) (float64, float64) {
    // Ищем угловую точку (x,y) ячейки (i,j)
    x := xyrange * (float64(i)/cells - 0.5)
    y := xyrange * (float64(j)/cells - 0.5)

    // Вычисляем высоту поверхности z
    z := f(x, y)

    // Изометрически проецируем (x,y,z) на двумерную канву SVG (sx,sy)
    sx := float64(width/2) + (x-y)*cos30*xyscale
    sy := float64(height/2) + (x+y)*sin30*xyscale - z*zscale
    return sx, sy
}

func f(x, y float64) float64 {
    r := math.Hypot(x, y) // Расстояние от (0,0)
    if r == 0 {
        return 0
    }
}
```

```
    return math.Sin(r) / r  
}
```



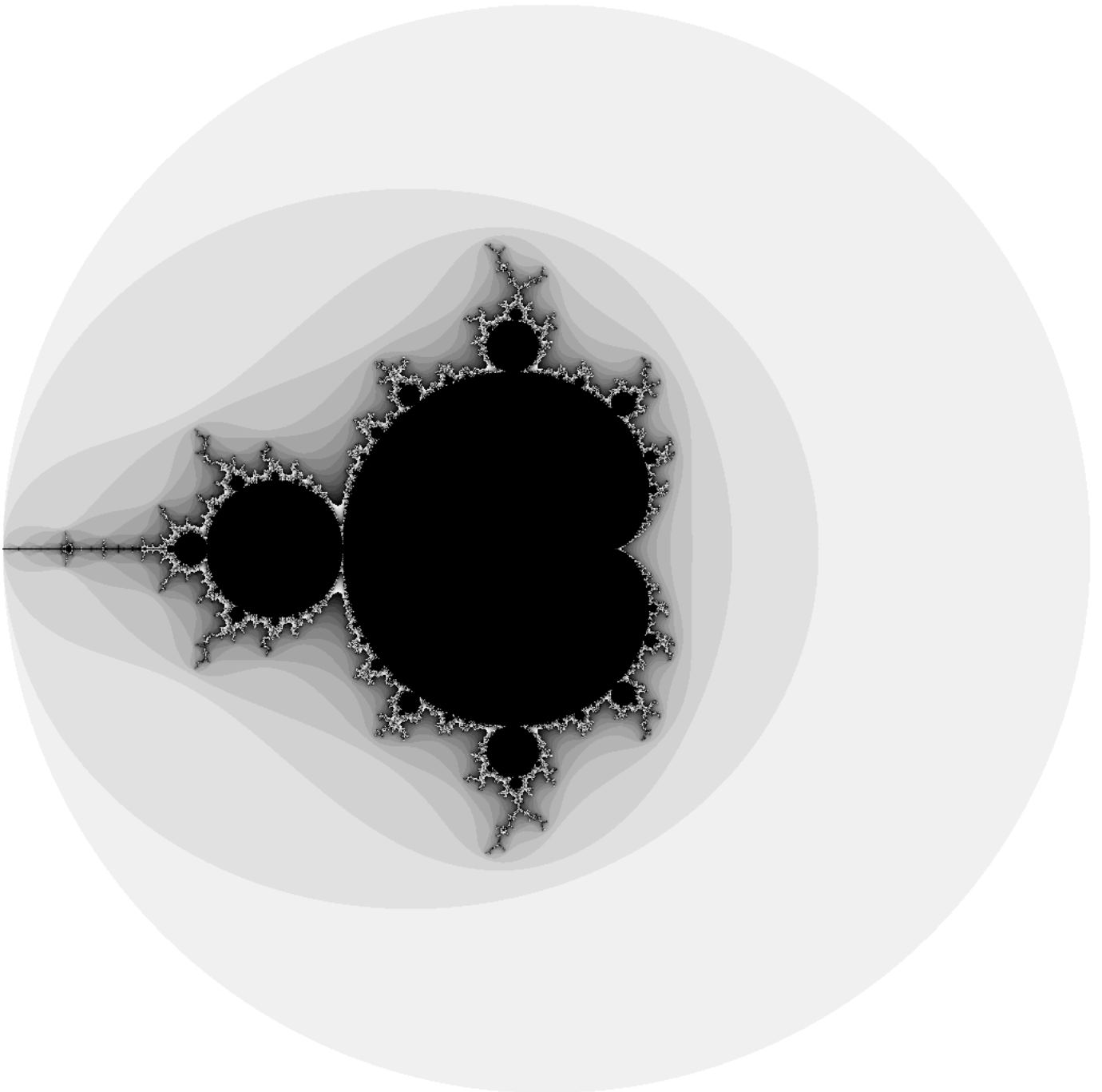
Complex Numbers

Go предоставляет комплексные числа двух размеров, `complex64` и `complex128`, компонентами которых являются `float32` и `float64` соответственно. Встроенная

функция `complex` создает комплексное число из действительной и мнимой компонент. а встроенные функции `real` и `imag` извлекают эти компоненты:

```
var x complex128 = complex(1, 2) // 1+2i
var y complex128 = complex(3, 4) // 3+4i
fmt.Println(x*y) // "(5+10i)"
fmt.Println(real(x*y)) // "5"
fmt.Println(imag(x*y)) // "10"
```

Чтож, создадим множество Мандельброта используя арифметику компл чисел:



```
package main

import (
    "image"
    "image/color"
    "image/png"
    "math/cmplx"
    "os"
)

func main() {
    const (
        xmin, ymin, xmax, ymax = -2, -2, +2, +2
        width, height           = 1024, 1024
    )

    img := image.NewRGBA(image.Rect(0, 0, width, height))
    for py := 0; py < height; py++ {
        y := float64(py)/height*(ymax-ymin) + ymin
        for px := 0; px < width; px++ {
            x := float64(px)/width*(xmax-xmin) + xmin
            z := complex(x, y)
            img.Set(px, py, mandelbrot(z))
        }
    }
    png.Encode(os.Stdout, img) // Примечание: игнорируем ошибки
}

func mandelbrot(z complex128) color.Color {
    const (
        iterations = 200
        contrast   = 15
    )

    var v complex128
    for n := uint8(0); n < iterations; n++ {
        v = v*v + z
        if cmplx.Abs(v) > 2 {
            return color.Gray{255 - contrast*n}
        }
    }
    return color.Black
}
```

Booleans

Тип `bool` или булев тип, имеет только два возможных значения – `true` и `false`. Булевыми являются условия в инструкциях `if` и `for`, а операторы сравнения `==` и `<` дают булев результат. Унарный оператор `!` представляет собой логическое отрицание, так что `!true` равно `false` (можно сказать, что

`(!true==false)==true`, хотя в качестве хорошего стиля мы всегда упрощаем излишне сложные булевы выражения `наподобие` упрощения `x==true` до `x`).

Булевы значения могут быть объединены с помощью операторов `&&` (И) и `||`

(ИЛИ), при вычислении которых используется сокращенное (shortcircuit) вычисление: если ответ определяется значением левого операнда, правый операнд не вычисляется. Это делает безопасными выражения, подобные следующему:

```
s != "" && s[0] == 'x'
```


Strings

Строка представляет собой неизменяемую последовательность байтов. Строки могут содержать произвольные данные, в том числе байты со значением 0, но обычно

они содержат удобочитаемый для человека текст. Традиционно текстовые строки интерпретируются как последовательности символов Unicode в кодировке UTF-8, о чем подробнее мы поговорим ниже.

В Go, в отличие от C, строки не заканчиваются нулевым символом (`\0`). Они хранятся как последовательность байтов с заранее известной длиной. Это значит, что нулевой байт (`\0`) – это такой же обычный байт, как и любой другой, и он может спокойно существовать внутри строки или среза байтов

Но надо быть аккуратным при взаимодействии с библиотеками на C.

Оператор `+` работает как оператор конкатенации, также есть операция получения подстроки `s[i:j]`

Управляющие символы:

Управляющий символ	Описание
<code>\a</code>	Звуковой сигнал
<code>\b</code>	Забой (backspace)
<code>\f</code>	Перевод страницы
<code>\n</code>	Новая строка
<code>\r</code>	Возврат каретки
<code>\t</code>	Табуляция
<code>\v</code>	Вертикальная табуляция
<code>\'</code>	Одинарная кавычка
<code>\"</code>	Двойные кавычки
<code>\\</code>	Обратная косая черта

Constants

Interfaces

Go. Chapter VII: Interfaces

NEDO-OFFSEC

Go. Chapter VII: Interfaces

Interfaces

Интерфейсные типы выражают обобщения или абстракции поведения других типов. С помощью обобщения интерфейсы позволяют писать более гибкие и адаптируемые функции, не привязанные к деталям одной конкретной реализации.

Interfaces as Contracts

В Go есть еще одна разновидность типов – тип интерфейса. Интерфейс является абстрактным типом. Он не раскрывает представление или внутреннюю структуру своих значений, или набор основных поддерживаемых ими операций.

Он показывает только некоторые из их методов.

Если у вас есть значение интерфейсного типа, вы **НИЧЕГО** не знаете о том, чем оно **ЯВЛЯЕТСЯ**, вы знаете **ТОЛЬКО** то, что оно может **ДЕЛАТЬ** или, точнее какое поведение предоставляется его методами

ПРИМЕР:

У нас есть 2 функции:

- `fmt.Printf`, которая записывает результаты в стандартный поток вывода
- `fmt.Sprintf`, которая возвращает результат в виде строки `string`

Обе функции являются, по сути, **ОБЕРТКАМИ** для функции `fmt.Fprintf`, которая **НИЧЕГО НЕ ЗНАЕТ** о том, что будет с результатом, который она вычисляет.

```
package fmt

func Fprintf(w io.Writer, format string, args ...interface{}) (int, error)

func Printf(format string, args ...interface{}) (int, error) {
    return Fprintf(os.Stdout, format, args...)
}

func Sprintf(format string, args ...interface{}) string {
    var buf bytes.Buffer
    Fprintf(&buf, format, args...)
    return buf.String()
}
```

Первый параметр `Fprintf` вообще не является файлом, его тип – `io.Writer`, который является типом интерфейса со следующим объявлением:

```
package io

type Writer interface {
    // Write записывает len(p) байтов из p в базовый поток данных.
    // Метод возвращает количество байтов, записанных из p
    // (0 <= n <= len(p)), а любая ошибка вызывает прекращение записи.
    // Write должен вернуть ненулевую ошибку при n < len(p).
    // Write не должен изменять данные среза, даже временно.
    //
    // Реализации не должны сохранять p.
    Write(p []byte) (n int, err error)
}
```

Интерфейс `io.Writer` определяет контракт между `Fprintf` и вызывающими его функциями. С одной стороны, контракт требует, чтобы вызывающая функция предоставляла значение конкретного типа наподобие `*os.File` или `*bytes.Buffer`, который содержит метод с именем `Write` с соотв. сигнатурой и поведением.

Таким образом, нам неважно что за тип передан в `w`. Нам важно только то, что у этого типа есть метод `Write` в этом и фишка интерфейсов. Интерфейс `Writer` это такой тип, у которого есть функция `Writer`, описанная выше

Поэтому мы и могли передать несколько разных типов в `Fprintf` потому у него и есть обертки

ПРИМЕР:

```
type Dog struct {
    Name string
}

func (d Dog) Speak() string {
    return "Woof!"
}

type Cat struct {
    Name string
}

func (c Cat) Speak() string {
    return "Meow!"
}

type Speaker interface {
    Speak() string
}

func Greet(s Speaker) {
    fmt.Println("Hello,", s.Speak())
}

func main() {
    dog := Dog{Name: "Buddy"}
    cat := Cat{Name: "Kitty"}

    Greet(dog) // Hello, Woof!
    Greet(cat) // Hello, Meow!
}
```

СРАВНЕНИЕ GO vs CPP:

Golang

```
type Speaker interface { Speak() }
type Dog struct {}

func (d Dog) Speak() { fmt.Println("Woof!") }

var s Speaker = Dog{}
s.Speak() // ← Вызов через ITable (позднее связывание)
```

C++

```
class Animal {
public:
    virtual void Speak() = 0;
};

class Dog : public Animal {
public:
    void Speak() override { cout << "Woof!"; }
};

Animal* a = new Dog();
a->Speak(); // ← Вызов через VTable (позднее связывание)
```

Таким образом, интерфейсы Go очень похожи на виртуальные функции C++.

Отличие Go интерфейсов от виртуальных ф-й C++:

Они работают на схожем механизме позднего связывания, через таблицы методов. Но ключевое отличие в том, что реализация интерфейса в Go происходит неявно. Мне не нужно наследоваться от интерфейса (implements). Если у типа есть все методы, он автоматически удовлетворяет интерфейсу. Это делает код более гибким и свободным от жёстких иерархий

ВАЖНО:

Позднее связывание – это связывание, которое определяется в run-time

Раннее связывание – это связывание, которое определяется в compile-time

Interface Types

Важные интерфейсы из пакета io

```
type Reader interface {
    Read(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}
```

Встраивание интерфейсов (Interface Embedding)

Интерфейсы можно встраивать друг в друга, как структуры.

```
type ReadWriter interface {
    Reader
    Writer
}

type ReadWriteCloser interface {
    Reader
    Writer
    Closer
}
```

Эквивалентно (без встраивания):

```
type ReadWriter interface {
    Read(p []byte) (n int, err error)
    Write(p []byte) (n int, err error)
}
```

Комбинация стилей

```
type ReadWriter interface {
    Read(p []byte) (n int, err error)
    Writer
}
```

Правила встраивания

Порядок методов не важен. Важно только их наличие.

Дублирование методов недопустимо. Если два встроенных интерфейса имеют методы с одинаковыми именами и сигнатурами, это нормально. Если сигнатуры разные – ошибка компиляции.

Встраивание работает только для интерфейсов. Встраивать структуры в интерфейсы **НЕЛЬЗЯ**.

Что важно запомнить

- Интерфейс определяет поведение.
- Конкретный тип реализует интерфейс неявно.
- Встраивание помогает строить сложные интерфейсы из простых.

Пример: fmt.Fprintf и io.Writer

```
func Fprintf(w io.Writer, format string, args ...interface{}) (int, error)
Fprintf не знает, что за тип передан в w:
```

- Это файл?
- Это буфер в памяти?
- Это сетевое соединение?
- Ему важно только одно: у типа есть метод Write.

Interface Satisfaction

Тип соответствует (удовлетворяет) интерфейсу, если он обладает всеми методами, которые требует интерфейс.

Правило присваиваемости интерфейсов очень простое:

выражение может быть присвоено интерфейсу, только если его тип соответствует этому интерфейсу.

Таким образом:

```
var w io.Writer
w = os.Stdout           // OK: *os.File имеет метод Write
w = new(bytes.Buffer)   // OK: *bytes.Buffer имеет метод Write
w = time.Second         // Ошибка: у time.Duration нет метода Write
```

```
var rwc io.ReadWriteCloser
rwc = os.Stdout         // OK: *os.File имеет методы Read, Write, Close
rwc = new(bytes.Buffer) // Ошибка: у *bytes.Buffer нет метода Close
```

Это правило применимо даже тогда, когда правая сторона сама по себе является интерфейсом:

```
w = rwc                 // OK: io.ReadWriteCloser имеет метод Write
rwc = w                 // Ошибка: io.Writer не имеет метода Close
```

Тонкий момент: Получатель (Pointer vs Value)

Метод с получателем-указателем (*T) и метод с получателем-значением (T) — это разные вещи.

Если метод объявлен как func (t *T) Method(), то интерфейсу удовлетворяет только *T.

Если метод объявлен как func (t T) Method(), то интерфейсу удовлетворяют и T, и *T.

ПРИМЕР:

```
type IntSet struct { /* ... */ }
func (*IntSet) String() string // метод с получателем-указателем

var s IntSet

var _ fmt.Stringer = &s // OK: *IntSet имеет метод String
var _ fmt.Stringer = s  // Ошибка: у IntSet нет метода String
```

Композиция интерфейсов

Если интерфейс A встроен в интерфейс B, то для удовлетворения B нужно реализовать все методы A и B.

```
go
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
type ReadWriter interface {
    Reader
    Writer
}
```

Пустой интерфейс interface{}

interface{} не имеет методов. Ему удовлетворяет любой тип.

```
var any interface{}
any = true
any = 42
any = "hello"
any = map[string]int{"one": 1}
```

flag.Value

Суть

Пакет `flag` позволяет создавать свои собственные флаги для командной строки. Для этого нужно реализовать интерфейс `flag.Value`:

```
package flag

type Value interface {
    String() string
    Set(string) error
}
```

- `String()` – как отображать значение флага.
- `Set(string)` – как парсить строку, переданную в флаг, и сохранять значение.

Пример: Свой флаг для температуры

Программа позволяет передавать температуру в формате “100C” или “212F” и преобразует её в градусы Цельсия.

Как это работает

```
type celsiusFlag struct{ Celsius } // Встраивание типа
func (f *celsiusFlag) Set(s string) error {
    var value float64
    var unit string
    fmt.Sscanf(s, "%f%s", &value, &unit) // Парсинг числа и буквы
    switch unit {
    case "C", "°C":
        f.Celsius = Celsius(value)
    case "F", "°F":
        f.Celsius = FToC(Fahrenheit(value))
    default:
        return fmt.Errorf("неверная температура %q", s)
    }
    return nil
}
```

Функция-конструктор

```
func CelsiusFlag(name string, value Celsius, usage string) *Celsius {
    f := celsiusFlag{value}
    flag.CommandLine.Var(&f, name, usage) // Регистрация в системе флагов
    return &f.Celsius
}
```

Использование

```
var temp = CelsiusFlag("temp", 20.0, "температура")
func main() {
    flag.Parse()
    fmt.Println(*temp) // Автоматически вызовется String()
}
```

Запуск

```
$ ./tempflag -temp -18C
-18°C

$ ./tempflag -temp 212°F
100°C
```

Ключевая мысль

Интерфейс `flag.Value` даёт возможность расширить стандартную библиотеку Go, не меняя её код. Достаточно определить свой тип и реализовать два метода – `String()` и `Set()`.

Interface Values

Устройство интерфейса

Значение интерфейса в Go – это **ПАКЕТ ИЗ ДВУХ ЭЛЕМЕНТОВ**:

1. **Динамический тип** (конкретный тип, который хранится внутри).
2. **Динамическое значение** (само значение этого типа).

Оно как коробка, в которой лежит что-то конкретное, и мы знаем, что это за вещь.

Нулевое значение интерфейса (nil)

```
var w io.Writer // w == nil, тип и значение = nil
```

У такого интерфейса нет ни типа, ни значения. Если вызвать его метод – программа упадёт (паника).

Присваивание конкретного типа интерфейсу

```
w = os.Stdout
```

Теперь внутри w:

Тип: `*os.File`

Значение: указатель на `os.Stdout`

Вызов `w.Write([]byte("hello"))` превратится в вызов `(*os.File).Write`.

Присваивание другого типа

```
w = new(bytes.Buffer)
```

Теперь внутри w:

Тип: `*bytes.Buffer`

Значение: указатель на новый буфер

Вызов `w.Write([]byte("hello"))` теперь пишет в буфер, а не в консоль.

Значение интерфейса равно nil только когда оба компонента равны nil

```
var w io.Writer // w == nil (тип nil, значение nil)
```

```
w = (*bytes.Buffer)(nil) // w != nil (тип *bytes.Buffer, значение nil)
```

Важно! Это частая ошибка. Если интерфейсу присвоить `nil`-указатель, он перестаёт быть `nil` интерфейсом.

Сравнение интерфейсов

Два интерфейса равны (`==`), если:

Оба `nil`.

Или их динамические типы совпадают, и динамические значения равны.

Осторожно: Если динамический тип несравним (например, `[]int` или `map`), сравнение вызовет панику.

```
var x interface{} = []int{1, 2, 3}
fmt.Println(x == x) // Паника! Срезы несравнимы.
```

Как узнать тип внутри интерфейса

```
var w io.Writer = os.Stdout
fmt.Printf("%T\n", w) // "*os.File"
```

Итог

- Интерфейс = (тип, значение).
- `nil` интерфейс = (`nil`, `nil`).
- Интерфейс с `nil`-указателем `!= nil` интерфейсу.
- Сравнивай интерфейсы осторожно – внутри может быть несравнимый тип.

Sorting with sort.Interface

Суть

В Go сортировка работает через интерфейс `sort.Interface`.

```
package sort

type Interface interface {
    Len() int           // длина последовательности
    Less(i, j int) bool // сравнение элементов i и j
    Swap(i, j int)      // обмен элементов i и j
}
```

Как это работает

Любой тип, у которого есть эти 3 метода, можно отсортировать функцией `sort.Sort`.

Пример: срез строк

```
type StringSlice []string

func (p StringSlice) Len() int           { return len(p) }
func (p StringSlice) Less(i, j int) bool { return p[i] < p[j] }
func (p StringSlice) Swap(i, j int)      { p[i], p[j] = p[j], p[i] }

// Использование
names := []string{"Moby", "Delilah", "Alicia"}
sort.Sort(StringSlice(names))
```

Для строк уже есть готовый `sort.Strings(names)`.

Сортировка структуры

Структура `Track`:

```
type Track struct {
    Title  string
    Artist string
    Album  string
    Year   int
    Length time.Duration
}
```

Чтобы отсортировать срез треков по исполнителю:

```
type byArtist []*Track

func (x byArtist) Len() int           { return len(x) }
func (x byArtist) Less(i, j int) bool { return x[i].Artist < x[j].Artist }
func (x byArtist) Swap(i, j int)      { x[i], x[j] = x[j], x[i] }

// Сортировка
sort.Sort(byArtist(tracks))
```

Обратная сортировка

```
sort.Sort(sort.Reverse(byArtist(tracks)))
```

`sort.Reverse` оборачивает любой `sort.Interface` и меняет логику `Less` на противоположную.

Многоуровневая сортировка

```
type customSort struct {
    t    []*Track
    less func(x, y *Track) bool
}

func (x customSort) Len() int           { return len(x.t) }
func (x customSort) Less(i, j int) bool { return x.less(x.t[i], x.t[j]) }
func (x customSort) Swap(i, j int)      { x.t[i], x.t[j] = x.t[j], x.t[i] }
```

Сортировка по нескольким полям:

```
sort.Sort(customSort{tracks, func(x, y *Track) bool {  
    if x.Title != y.Title {  
        return x.Title < y.Title  
    }  
    if x.Year != y.Year {  
        return x.Year < y.Year  
    }  
    return x.Length < y.Length  
}})
```

Готовые функции

Пакет sort уже умеет сортировать:

```
sort.Ints([]int{3, 1, 4})           // []int  
sort.Strings([]string{"b", "a"})   // []string  
sort.Float64s([]float64{3.14, 1.0}) // []float64
```

Ключевая мысль

sort.Interface – это абстракция. Она работает с любой последовательностью: срез, массив, структура.

Главное – реализовать 3 метода: Len, Less, Swap.

HTTP Handlers

Суть

В Go веб-сервер строится на интерфейсе `http.Handler`:

```
package http
```

```
type Handler interface {  
    ServeHTTP(w ResponseWriter, r *Request)  
}
```

Любой тип, у которого есть метод `ServeHTTP`, является обработчиком HTTP-запросов.

Простейший сервер

```
type database map[string]float64  
  
func (db database) ServeHTTP(w http.ResponseWriter, r *http.Request) {  
    for item, price := range db {  
        fmt.Fprintf(w, "%s: %.2f\n", item, price)  
    }  
}  
  
func main() {  
    db := database{"shoes": 50, "socks": 5}  
    log.Fatal(http.ListenAndServe("localhost:8000", db))  
}
```

Сервер принимает любой запрос и выводит все товары.

Маршрутизация по URL

```
func (db database) ServeHTTP(w http.ResponseWriter, r *http.Request) {  
    switch r.URL.Path {  
    case "/list":  
        for item, price := range db {  
            fmt.Fprintf(w, "%s: %.2f\n", item, price)  
        }  
    case "/price":  
        item := r.URL.Query().Get("item")  
        price, ok := db[item]  
        if !ok {  
            http.Error(w, "not found", http.StatusNotFound)  
            return  
        }  
        fmt.Fprintf(w, "%.2f\n", price)  
    default:  
        http.Error(w, "not found", http.StatusNotFound)  
    }  
}
```

Использование ServeMux (мультиплексор)

```
func main() {  
    db := database{"shoes": 50, "socks": 5}  
    mux := http.NewServeMux()  
    mux.Handle("/list", http.HandlerFunc(db.list))  
    mux.Handle("/price", http.HandlerFunc(db.price))  
    log.Fatal(http.ListenAndServe("localhost:8000", mux))  
}
```

`ServeMux` – это роутер. Он сопоставляет URL с обработчиками.

Функция как обработчик (`http.HandlerFunc`)

```
type HandlerFunc func(w ResponseWriter, r *Request)
```

```
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) {  
    f(w, r)  
}
```

`HandlerFunc` -- это адаптер. Любая функция с сигнатурой func(w http.ResponseWriter, r *http.Request)` может стать http.Handler` через преобразование типа.`

```
mux.Handle("/list", http.HandlerFunc(db.list)) // явно
mux.HandleFunc("/list", db.list)              // сокращённо
```

Глобальный DefaultServeMux

```
func main() {
    db := database{"shoes": 50, "socks": 5}
    http.HandleFunc("/list", db.list) // регистрирует в глобальном роутере
    http.HandleFunc("/price", db.price)
    log.Fatal(http.ListenAndServe("localhost:8000", nil)) // nil = DefaultServeMux
}
```

Важно!

Каждый HTTP-запрос обрабатывается в своей go-подпрограмме. Если обработчик меняет общие данные — используй `sync.Mutex`, чтобы избежать гонок.

Ключевая мысль

Интерфейс `http.Handler` и `ServeMux` делают веб-серверы гибкими. Можно регистрировать любые обработчики, не меняя структуру сервера.

Error Interface

Суть

В Go ошибка – это не магия. Это просто интерфейс:

```
type error interface {  
    Error() string  
}
```

Любой тип, у которого есть метод `Error() string`, автоматически является ошибкой.

Как создать ошибку

```
import "errors"  
  
err := errors.New("something went wrong")
```

Внутри `errors.New` возвращает указатель на структуру `errorString`, которая реализует `error`.

```
package errors  
  
type errorString struct { text string }  
  
func (e *errorString) Error() string { return e.text }
```

Почему там указатель, а не значение?

Чтобы две ошибки с одинаковым текстом не были равны:

```
fmt.Println(errors.New("EOF") == errors.New("EOF")) // false
```

Если бы использовалась структура, а не указатель, они могли бы быть равны, и это нарушило бы логику работы с `io.EOF`.

Форматирование ошибок

```
import "fmt"  
  
err := fmt.Errorf("file %s not found", filename)
```

`fmt.Errorf` – это обёртка над `errors.New`, которая сначала форматирует строку.

Пример: системная ошибка

```
package syscall  
  
type Errno uintptr  
  
var errors = [...]string{  
    1: "operation not permitted",  
    2: "no such file or directory",  
}  
  
func (e Errno) Error() string {  
    if int(e) < len(errors) {  
        return errors[e]  
    }  
    return fmt.Sprintf("errno %d", e)  
}
```

Теперь `syscall.Errno(2)` – это ошибка с текстом “no such file or directory”.

Ключевая мысль

Интерфейс `error` – это просто контракт на метод `Error()`. Это даёт гибкость: можно возвращать ошибки разных типов, но работать с ними через один интерфейс.

Type Assertions

Суть

Декларация типа (type assertion) — это операция, которая извлекает значение из интерфейса.

Синтаксис: `x.(T)`

- `x` — значение интерфейсного типа
- `T` — декларируемый тип

Два варианта

Вариант 1: Конкретный тип

Если `T` — конкретный тип, то `x.(T)` проверяет, хранит ли `x` именно этот тип.

```
var w io.Writer
w = os.Stdout

f := w.(*os.File) // OK: f == os.Stdout
c := w.(*bytes.Buffer) // Паника: интерфейс хранит *os.File, а не *bytes.Buffer
```

Вариант 2: Интерфейс

Если `T` — интерфейс, то `x.(T)` проверяет, реализует ли динамический тип `x` интерфейс `T`.

```
var w io.Writer = os.Stdout
rw := w.(io.ReadWriter) // OK: *os.File реализует io.ReadWriter
```

Безопасная форма (с проверкой)

```
f, ok := w.(*os.File)
if ok {
    // f — *os.File, можно использовать
} else {
    // w хранит не *os.File
}
```

Если декларация неудачна, `ok == false`, а значение — нулевое для этого типа.

Пример: распознавание ошибок

Пакет `os` определяет структуру `PathError`:

```
type PathError struct {
    Op    string
    Path  string
    Err   error
}

func (e *PathError) Error() string {
    return e.Op + " " + e.Path + ": " + e.Err.Error()
}
```

Функция `os.IsNotExist` использует декларацию типа для проверки:

```
func IsNotExist(err error) bool {
    if pe, ok := err.(*PathError); ok {
        err = pe.Err
    }
    return err == syscall.ENOENT || err == ErrNotExist
}
```

Теперь можно отличить “файл не найден” от других ошибок.

```
err := os.Open("/no/such/file")
fmt.Println(os.IsNotExist(err)) // true
```

Ключевая мысль

Декларация типа (`x.(T)`) — это способ заглянуть внутрь интерфейса и получить конкретное значение или проверить, какой тип там хранится. Используй безопасную форму с `ok`, чтобы избежать паники.

Type Switches

Два стиля интерфейсов

1. **Полиморфизм подтипов** – интерфейс определяет поведение (методы). Примеры: `io.Reader`, `http.Handler`, `sort.Interface`.
2. **Распознаваемое объединение** – интерфейс хранит один из множества типов. Акцент на том, **какой** это тип, а не на методах.

Пример: sqlQuote

Функция преобразует значение любого типа в SQL-литерал:

```
func sqlQuote(x interface{}) string {
    switch x := x.(type) {
    case nil:
        return "NULL"
    case int, uint:
        return fmt.Sprintf("%d", x)    // x – interface{}, но мы знаем, что это число
    case bool:
        if x {
            return "TRUE"
        }
        return "FALSE"
    case string:
        return sqlQuoteString(x)       // x – string
    default:
        panic(fmt.Sprintf("unexpected type %T: %v", x, x))
    }
}
```

Синтаксис

```
switch x := x.(type) {
case nil:
    // x – interface{} (nil)
case int:
    // x – int
case string:
    // x – string
case bool, uint:
    // x – interface{} (несколько типов в одном case)
default:
    // x – interface{}
}
```

Как это работает

`x.(type)` – это специальная форма для `switch`.

В каждом `case` переменная `x` имеет конкретный тип этого `case`.

Если в одном `case` несколько типов (например, `int`, `uint`), то `x` остаётся `interface{}`.

Порядок важен

Если один тип является интерфейсом, он может “поглотить” другие. Например:

```
switch x.(type) {
case fmt.Stringer:
    // ...
case string:
    // ...
}
```

Здесь `string` никогда не выполнится, потому что `string` реализует `fmt.Stringer`.

Где используется

- Парсинг JSON/YAML.
- Обработка SQL-запросов (как в примере).
- Логирование значений разных типов.

- Работа с `interface{}` в обобщённых функциях.

Ключевая мысль

Выбор типа – это способ заглянуть внутрь `interface{}` и сделать разную логику для разных типов. Это как `switch`, но не по значению, а по типу.

Gorouties and channels

Go. Chapter VIII: Goroutines and Channels

NEDO-OFFSEC

Go. Chapter VIII: Goroutines and Channels

Goroutines and channels

Go обеспечивает два стиля параллельного программирования. В этой главе представлены go-подпрограммы (goroutines) и каналы, которые поддерживают взаимодействующие последовательные процессы (communicating sequential processes — CSP),

модель параллелизма, в которой между независимыми процессами (go-подпрограммами) передаются значения, но переменные по большей части ограничиваются одним процессом.

Goroutines

Что это

Горутина (goroutine) – это лёгкий поток выполнения в Go. Она запускается ключевым словом `go`.

```
f()      // обычный вызов -- ждём завершения
go f()   // запуск в новой горутине -- не ждём
```

При запуске программы есть одна горутина – `main`. Когда `main` завершается, все остальные горутинны завершаются принудительно.

Пример: анимация во время вычислений

```
func main() {
    go spinner(100 * time.Millisecond) // запускаем анимацию в фоне
    fibN := fib(45)                    // долгое вычисление
    fmt.Printf("Fibonacci(%d) = %d\n", 45, fibN)
}

func spinner(delay time.Duration) {
    for {
        for _, r := range `-\|/` {
            fmt.Printf("\r%c", r)
            time.Sleep(delay)
        }
    }
}

func fib(x int) int {
    if x < 2 {
        return x
    }
    return fib(x-1) + fib(x-2)
}
```

Что здесь происходит

`go spinner(...)` запускает анимацию в фоне.

`fib(45)` выполняется одновременно.

Когда `main` завершится – программа заканчивается, даже если `spinner` ещё работает.

Важно

Горутинны – это не потоки ОС. Они легче: стек растёт динамически, планировщик Go управляет ими.

Синтаксис

```
go func() {
    // код
}()
```

Анонимную функцию тоже можно запустить как горутину.

Ключевая мысль

`go` – это способ запустить функцию параллельно с основной программой. Программа завершается, когда завершается `main`, независимо от того, работают ли другие горутины.

EXAMPLE: Concurrent echo-server

Простейший эхо-сервер (синхронный)

```
func handleConn(c net.Conn) {
    io.Copy(c, c) // читает и сразу пишет обратно
    c.Close()
}
```

Эхо с задержкой (синхронный)

```
func echo(c net.Conn, shout string, delay time.Duration) {
    fmt.Fprintln(c, "\t", strings.ToUpper(shout))
    time.Sleep(delay)
    fmt.Fprintln(c, "\t", shout)
    time.Sleep(delay)
    fmt.Fprintln(c, "\t", strings.ToLower(shout))
}
```

```
func handleConn(c net.Conn) {
    input := bufio.NewScanner(c)
    for input.Scan() {
        echo(c, input.Text(), 1*time.Second) // синхронно
    }
    c.Close()
}
```

Проблема: пока обрабатывается один “крик”, новый ввод не принимается.

Параллельный эхо-сервер

Достаточно добавить `go` перед вызовом `echo`:

```
func handleConn(c net.Conn) {
    input := bufio.NewScanner(c)
    for input.Scan() {
        go echo(c, input.Text(), 1*time.Second) // асинхронно
    }
    c.Close()
}
```

Теперь каждый “крик” обрабатывается в своей горутине. Ответы приходят независимо друг от друга.

Клиент

```
func main() {
    conn, err := net.Dial("tcp", "localhost:8000")
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()

    go mustCopy(os.Stdout, conn) // читаем ответ сервера
    mustCopy(conn, os.Stdin)    // отправляем ввод на сервер
}
```

Одна горутина читает ответы, другая отправляет запросы – параллельно.

Важно!

При вызове `go echo(...)` аргументы вычисляются в момент вызова, но сама функция запускается асинхронно.

Ключевая мысль

Одно слово `go` превращает синхронный сервер в параллельный. Горутин позволяют обрабатывать несколько соединений и несколько операций внутри одного соединения одновременно.

Channels

Канал – это механизм связи между горутинами. Он позволяет передавать значения определённого типа.

```
ch := make(chan int) // канал для int
```

Основные операции

```
ch <- x    // отправка
x = <-ch   // получение
<-ch      // получение (игнорируем значение)
```

Небуферизованный канал

Отправка блокируется, пока кто-то не прочтёт. Получение блокируется, пока кто-то не отправит.

```
ch := make(chan int)
go func() { ch <- 42 }()
x := <-ch // 42
```

Буферизованный канал

```
ch := make(chan string, 2)
ch <- "hello"
ch <- "world"
fmt.Println(<-ch) // hello
fmt.Println(<-ch) // world
```

Отправка блокируется только когда буфер заполнен. Получение блокируется когда буфер пуст.

Закрытие канала

```
close(ch)
```

- После закрытия: чтение возвращает значение + `ok == false`
- Запись в закрытый канал вызывает панику

Однонаправленные каналы

```
var ch1 chan<- int // только отправка
var ch2 <-chan int // только получение
```

Конвейер (pipeline)

```
func main() {
    naturals := make(chan int)
    squares := make(chan int)

    go func() {
        for x := 0; x < 10; x++ {
            naturals <- x
        }
        close(naturals)
    }()

    go func() {
        for x := range naturals {
            squares <- x * x
        }
        close(squares)
    }()

    for x := range squares {
        fmt.Println(x)
    }
}
```

Итог

- Каналы – главный способ общения между горутинами.
- Небуферизованные – синхронные.
- Буферизованные – асинхронные (до заполнения буфера).
- Закрытие канала – сигнал, что данных больше не будет.

Select

`select` позволяет ждать несколько операций с каналами одновременно.

```
select {
case <-ch1:
    // ch1 готов
case x := <-ch2:
    // ch2 готов, x — полученное значение
case ch3 <- y:
    // можно отправить в ch3
default:
    // ничего не готово (неблокирующий режим)
}
```

Как работает

- `select` блокируется, пока один из каналов не станет готов.
- Если готово несколько — выбирает случайный.
- Если есть `default` — не блокируется.

Пример: таймер и отмена

```
func main() {
    abort := make(chan struct{})
    go func() {
        os.Stdin.Read(make([]byte, 1))
        abort <- struct{}{}
    }()

    fmt.Println("Обратный отсчёт. Enter — отмена.")
    select {
    case <-time.After(10 * time.Second):
        launch()
    case <-abort:
        fmt.Println("Отменено!")
    }
}
```

Таймеры

```
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()
```

```
for countdown := 10; countdown > 0; countdown-- {
    fmt.Println(countdown)
    select {
    case <-ticker.C:
        // прошла секунда
    case <-abort:
        fmt.Println("Отменено!")
        return
    }
}
```

`time.Tick` удобен, но создаёт утечку горутины, если не остановить. Используйте `time.NewTicker` и `ticker.Stop()`.

Неблокирующий select (с default)

```
select {
case <-abort:
    fmt.Println("Отменено!")
    return
default:
    // ничего не делаем, продолжаем
}
```

nil-каналы

Операции с `nil`-каналом блокируются навсегда. Это позволяет включать/выключать варианты в `select`.

Ключевая мысль

`select` – это `switch` для каналов. Он позволяет ждать несколько событий одновременно и реагировать на первое готовое.

Cancellation

Проблема

Нет способа напрямую остановить горутину из другой горутины. Нужен механизм широковещания.

Решение: закрытие канала

Когда канал закрыт, все читающие получают нулевое значение. Это работает как широковещательный сигнал.

```
done := make(chan struct{}) // канал для сигнала отмены
close(done)                 // сигнал всем
```

Проверка отмены

```
func cancelled() bool {
    select {
    case <-done:
        return true
    default:
        return false
    }
}
```

Пример: отмена по нажатию Enter

```
done := make(chan struct{})

go func() {
    os.Stdin.Read(make([]byte, 1)) // ждём Enter
    close(done)                    // сигнал всем
}()

// В горутинах:
select {
case <-done:
    return // отмена
default:
    // продолжать работу
}
```

Обработка отмены в цикле

```
select {
case <-done:
    for range fileSizes {
        // опустошаем канал, чтобы горутины не заблокировались
    }
    return
case size, ok := <-fileSizes:
    // обработка размера файла
}
```

Как это работает

1. Создаём канал done типа chan struct{}
2. Закрываем его при отмене
3. Все горутины проверяют done в ключевых точках
4. При отмене – выходят из работы

Ключевая мысль

Закрытие канала – это способ отправить сигнал всем горутинам одновременно. Это дешевле и надёжнее, чем отправлять отдельные сообщения.

EXAMPLE: Chat Server

Архитектура

Чат-сервер использует 4 типа горутин:

1. `main` — принимает подключения
2. `broadcaster` — управляет клиентами и рассылает сообщения
3. `handleConn` — обрабатывает одного клиента
4. `clientWriter` — отправляет сообщения клиенту

Каналы

```
type client chan<- string // канал для отправки сообщений клиенту
```

```
var (  
    entering = make(chan client) // новый клиент  
    leaving  = make(chan client) // клиент уходит  
    messages = make(chan string) // входящие сообщения  
)
```

Broadcaster

```
func broadcaster() {  
    clients := make(map[client]bool) // все подключённые клиенты  
  
    for {  
        select {  
        case msg := <-messages:  
            // отправить сообщение всем клиентам  
            for cli := range clients {  
                cli <- msg  
            }  
        case cli := <-entering:  
            clients[cli] = true  
        case cli := <-leaving:  
            delete(clients, cli)  
            close(cli)  
        }  
    }  
}
```

Обработка клиента

```
func handleConn(conn net.Conn) {  
    ch := make(chan string)  
    go clientWriter(conn, ch)  
  
    who := conn.RemoteAddr().String()  
    ch <- "Вы " + who  
    messages <- who + " подключился"  
    entering <- ch  
  
    input := bufio.NewScanner(conn)  
    for input.Scan() {  
        messages <- who + ": " + input.Text()  
    }  
  
    leaving <- ch  
    messages <- who + " отключился"  
    conn.Close()  
}  
  
func clientWriter(conn net.Conn, ch <-chan string) {  
    for msg := range ch {  
        fmt.Fprintln(conn, msg)  
    }  
}
```

Как это работает

1. `main` принимает подключения, запускает `handleConn`

2. `handleConn` создаёт канал для клиента, регистрирует его через `entering`
3. Клиент читает сообщения через `clientWriter`
4. Каждое сообщение отправляется в `messages`
5. `broadcaster` получает сообщение и отправляет его всем клиентам
6. При отключении клиент уведомляет через `leaving`

Ключевая мысль

Чат-сервер – это классический пример параллельного приложения, где горутины и каналы используются для управления множеством клиентов без явных блокировок.

Concurrency

Go. Chapter IX: Concurrency with Shared Variables

NEDO-OFFSEC

Go. Chapter IX: Concurrency with Shared Variables

Concurrency

В предыдущей главе мы представили несколько программ, которые используют go-подпрограммы и каналы для выражения параллелизма непосредственным и естественным путем. Однако, поступая так, мы затушевывали целый ряд важных и тонких вопросов, которые программисты должны учитывать при написании параллельного кода.

В этой главе мы более детально рассмотрим механику параллелизма. В частности, мы укажем на некоторые из проблем, связанных с совместным использованием переменных несколькими go-подпрограммами, и рассмотрим аналитические методы распознавания этих проблем и шаблоны их решений. Наконец мы поясним некоторые технические различия между go-подпрограммами и потоками операционной системы.

Data Races

Что такое состояние гонки (race condition)

Состояние гонки – ситуация, когда программа даёт неправильный результат при некотором чередовании операций нескольких горутин.

Гонка данных (data race)

Гонка данных возникает, когда две горутины одновременно обращаются к одной переменной, и хотя бы одно обращение – запись.

Пример: банковский счёт

```
var balance int

func Deposit(amount int) {
    balance = balance + amount // ЧТЕНИЕ + ЗАПИСЬ
}

func Balance() int {
    return balance
}
```

Проблема: `balance = balance + amount` – это две операции:

- Чтение `balance`
- Запись новой суммы
- Если две горутины вызовут `Deposit` одновременно, может произойти гонка.

Как это выглядит

Горутина А: читает `balance (0)` -> `balance + 200 = 200` Горутина Б: читает `balance (0)` -> `balance + 100 = 100`
Горутина А: записывает 200 Горутина Б: записывает 100 Результат: 100, хотя должно быть 300.

Три способа избежать гонки

- Не записывать переменную – только чтение.
- Ограничить переменную одной горутиной (например, через каналы).
- Взаимное исключение – допускать доступ только одной горутине за раз (Mutex).

Пример: ограничение через каналы

```
var deposits = make(chan int)
var balances = make(chan int)

func Deposit(amount int) { deposits <- amount }
func Balance() int      { return <-balances }

func teller() {
    var balance int
```

```
    for {
        select {
            case amount := <-deposits:
                balance += amount
            case balances <- balance:
                }
        }
    }

func init() {
    go teller()
}
```

Ключевая мысль

Гонка данных – это всегда ошибка. Нет “доброкачественных” гонок. Избегай их тремя способами: не пиши, ограничь горутины, или используй мьютексы.

Mutual Exclusion (sync.Mutex)

Проблема

Гонка данных возникает, когда несколько горутин одновременно обращаются к одной переменной, и хотя бы одна из них пишет.

Решение: sync.Mutex

Мьютекс обеспечивает взаимное исключение – доступ к критическому разделу получает только одна горутина за раз.

```
import "sync"

var mu sync.Mutex
var balance int

func Deposit(amount int) {
    mu.Lock()           // захват блокировки
    balance += amount
    mu.Unlock()         // освобождение блокировки
}

func Balance() int {
    mu.Lock()
    defer mu.Unlock()   // освобождение через defer
    return balance
}
```

Критический раздел (critical section)

Область между Lock() и Unlock() – это критический раздел. В нём горутина может безопасно читать и изменять защищаемые переменные.

defer для Unlock()

Используй defer mu.Unlock() сразу после Lock() – это гарантирует освобождение блокировки даже при панике или раннем возврате.

Мьютекс не реентерабельный

Мьютекс нельзя заблокировать повторно в той же горутине. Это приведёт к взаимоблокировке (deadlock).

```
func Withdraw(amount int) bool {
    mu.Lock()
    defer mu.Unlock()
    Deposit(-amount) // deadlock! Deposit пытается захватить тот же мьютекс
    // ...
}
```

Решение: разделить логику на внутреннюю функцию (без блокировки) и внешнюю (с блокировкой).

Паттерн: монитор

Набор экспортируемых функций, инкапсулирующих переменные и выполняющих блокировку/освобождение.

Ключевая мысль

Мьютекс – это простой и надёжный способ защитить общие данные. Главное правило: всегда освобождай блокировку через defer.

Read/Write Mutex (`sync.RWMutex`)

Проблема

Когда много горутин читают данные, а пишут редко, обычный `sync.Mutex` блокирует всех, даже читателей, хотя они не мешают друг другу.

Решение `sync.RWMutex`

`RWMutex` позволяет:

- Несколько читателей одновременно (`RLock`)
- Только одного писателя за раз (`Lock`)

```
var mu sync.RWMutex
var balance int

func Balance() int {
    mu.RLock()           // блокировка для чтения
    defer mu.RUnlock()
    return balance
}

func Deposit(amount int) {
    mu.Lock()            // исключительная блокировка
    defer mu.Unlock()
    balance += amount
}
```

Правила

- `RLock()` / `RUnlock()` – для читателей.
- `Lock()` / `Unlock()` – для писателей.

Читатели не блокируют друг друга. Писатель блокирует всех (и читателей, и других писателей).

Когда использовать

`RWMutex` имеет смысл, если:

1. Чтений гораздо больше, чем записей.
2. Блокировка – узкое место (горутины часто ждут).

`RWMutex` медленнее обычного `sync.Mutex` из-за более сложной логики. Не используй его без необходимости.

Важно

`RLock()` можно использовать только когда критический раздел точно не пишет в общие данные. Если сомневаешься – используй обычный `Lock()`.

Ключевая мысль

`RWMutex` – это оптимизация для сценариев с большим количеством чтений и редкими записями. Для остальных случаев используй обычный `sync.Mutex`.

Memory Synchronization

Проблема

В современных компьютерах несколько процессоров имеют собственные кеши. Записи в память могут буферизироваться и сбрасываться в разном порядке.

Без синхронизации горутина может читать устаревшие значения, даже если другая горутина уже записала новые.

Пример: недетерминированное поведение

```
var x, y int

go func() {
    x = 1
    fmt.Print("y:", y)
}()

go func() {
    y = 1
    fmt.Print("x:", x)
}()
```

Возможные выводы:

- y:0 x:1
- x:0 y:1
- x:1 y:1
- x:0 y:0

Почему так происходит

В отсутствие синхронизации:

- Компилятор может переставлять инструкции.
- Процессор может переставлять записи в память.
- Кеши разных ядер могут не синхронизироваться.

Решение

Синхронизация гарантирует порядок памяти.

Примитивы синхронизации (каналы, мьютексы) заставляют процессор сбросить все накопленные записи в основную память.

Правила

1. В пределах одной горутины порядок гарантирован.
2. Между горутинами – только через синхронизацию.

Ключевая мысль

Синхронизация нужна не только для порядка выполнения, но и для видимости изменений в памяти. Без неё программа может вести себя недетерминированно.

sync.Once

Проблема

Нужно выполнить инициализацию только один раз, но функция может быть вызвана параллельно из многих горутин.

Простая проверка `if icons == nil` не работает из-за гонок данных и переупорядочивания памяти.

Решение: sync.Once

```
var loadIconsOnce sync.Once
var icons map[string]image.Image

func Icon(name string) image.Image {
    loadIconsOnce.Do(loadIcons)
    return icons[name]
}
```

Как работает Do

- Первый вызов `Do(f)` выполняет `f()`.
- Последующие вызовы `Do(f)` ничего не делают.
- `Do` гарантирует, что `f()` выполнится только один раз, даже при параллельных вызовах.
- `Do` блокирует вызовы до завершения `f()`.

Почему лучше, чем мьютекс

Подход	Проблема
<code>sync.Mutex</code>	Блокирует всех даже после инициализации
<code>sync.RWMutex</code>	Сложный код, нужно перепроверять <code>nil</code>
<code>sync.Once</code>	Простой, безопасный, эффективный

Итог

`sync.Once` – идеальный инструмент для ленивой инициализации. Гарантирует, что функция выполнится ровно один раз, даже при параллельных вызовах, и все горутины увидят результат.

Race Detector

Что это

Детектор гонки – инструмент в Go, который находит гонки данных во время выполнения программы.

Как использовать

Добавь флаг `-race` к командам `go build`, `go run` или `go test`:

```
go run -race main.go
go build -race main.go
go test -race ./...
```

Как работает

1. Компилятор создаёт специальную версию программы.
2. Программа записывает все обращения к переменным и события синхронизации (каналы, мьютексы, горутины).
3. Детектор ищет одновременный доступ к одной переменной без синхронизации.
4. При обнаружении выводит отчёт с указанием места и стеков вызовов.

Пример отчёта

```
WARNING: DATA RACE
Write at 0x00c0000a00c0 by goroutine 7:
    main.Deposit()
    bank.go:10 +0x5a
```

```
Previous read at 0x00c0000a00c0 by goroutine 8:
    main.Balance()
    bank.go:15 +0x4a
```

Ограничения

- Детектор находит только те гонки, которые реально произошли во время выполнения.
- Не может найти гонки, которые не были запущены тестами.
- Программа с `-race` работает медленнее и требует больше памяти.

Ключевая мысль

Детектор гонки – это обязательный инструмент при разработке параллельных программ. Используй его в тестах и при отладке.

Concurrent Cache

Проблема

Нужно кешировать результаты дорогих вызовов функций (например, HTTP-запросов). Кеш должен быть безопасным при параллельном доступе и не блокировать другие запросы.

Эволюция дизайна

Версия 1: Простой кеш (небезопасный)

```
type Memo struct {
    f      Func
    cache map[string]result
}

func (memo *Memo) Get(key string) (interface{}, error) {
    res, ok := memo.cache[key]
    if !ok {
        res.value, res.err = memo.f(key)
        memo.cache[key] = res
    }
    return res.value, res.err
}
```

Гонка данных при параллельном доступе.

Версия 2: С мьютексом (безопасно, но медленно)

```
type Memo struct {
    f      Func
    mu     sync.Mutex
    cache map[string]result
}

func (memo *Memo) Get(key string) (interface{}, error) {
    memo.mu.Lock()
    defer memo.mu.Unlock()
    // ... та же логика
}
```

Безопасно. Блокирует все вызовы, даже уже закешированные.

Версия 3: Двойная блокировка (быстрее, но дублирует работу)

```
func (memo *Memo) Get(key string) (interface{}, error) {
    memo.mu.Lock()
    res, ok := memo.cache[key]
    memo.mu.Unlock()

    if !ok {
        res.value, res.err = memo.f(key)
        memo.mu.Lock()
        memo.cache[key] = res
        memo.mu.Unlock()
    }
    return res.value, res.err
}
```

Быстрее. Дублирует вычисления (две горутины могут вычислять один ключ).

Версия 4: Подавление повторений (без дублирования)

```
type entry struct {
    res result
    ready chan struct{} // закрывается, когда res готов
}

type Memo struct {
    f      Func
    mu     sync.Mutex
    cache map[string]*entry
}
```



```
}

func (memo *Memo) Get(key string) (interface{}, error) {
    memo.mu.Lock()
    e := memo.cache[key]
    if e == nil {
        e = &entry{ready: make(chan struct{})}
        memo.cache[key] = e
        memo.mu.Unlock()

        e.res.value, e.res.err = memo.f(key)
        close(e.ready) // СИГНАЛ ГОТОВНОСТИ
    } else {
        memo.mu.Unlock()
        <-e.ready // ждём готовности
    }
    return e.res.value, e.res.err
}
```

Безопасно, быстро, без дублирования.

Монитор (управляющая горутина)

Альтернативный подход — ограничить кеш одной горутиной и общаться через каналы.

```
type request struct {
    key      string
    response chan result
}

func (memo *Memo) server(f Func) {
    cache := make(map[string]*entry)
    for req := range memo.requests {
        e := cache[req.key]
        if e == nil {
            e = &entry{ready: make(chan struct{})}
            cache[req.key] = e
            go e.call(f, req.key)
        }
        go e.deliver(req.response)
    }
}
```

Ключевая мысль

Параллельный кеш – это классическая задача, показывающая эволюцию дизайна:

- Наивное решение (гонки)
- Безопасное решение (мьютекс)
- Оптимизированное решение (подавление повторений)
- Альтернативный подход (монитор через каналы)

Goroutines vs Threads

Стеки

Потоки ОС	Горутин
Фиксированный размер (обычно 2 МБ)	Динамический, начинается с 2 КБ
Трата памяти даже для простых задач	Растёт и уменьшается по мере необходимости
Нельзя иметь сотни тысяч	Можно иметь сотни тысяч одновременно

Планирование

Потоки ОС	Горутин
Планируются ядром ОС	Планируются рантаймом Go (M:N планирование)
Переключение контекста – дорого	Переключение контекста — дешево
Прерывание по таймеру	Неявное переключение при блокировке (каналы, мьютексы, <code>time.Sleep</code>)

GOMAXPROCS

GOMAXPROCS определяет, сколько потоков ОС одновременно выполняют код Go.

По умолчанию = количество ядер процессора.

```
runtime.GOMAXPROCS(1) // только один поток
runtime.GOMAXPROCS(8) // 8 потоков
```

Идентификация горутин

- У горутин нет идентификатора, доступного программисту.
- Это сделано намеренно – чтобы избежать “локальной памяти потока” (Thread-Local Storage).
- Вместо этого все параметры передаются явно.
- Код становится чище, понятнее и проще для тестирования.

Ключевая мысль

Горутин легче, быстрее и масштабируемее потоков ОС. Их планирование управляется рантаймом Go, что делает параллельное программирование эффективным и доступным.